

Internet & Web Programming

Midterm study notes, built from Assignments 01–12.

Md. Mahir Musleh — Institute of Information Technology, University of Dhaka

Built from Assignments 01–12. Each section states the theory, then points at the assignment where you already wrote it, then lists likely exam questions.

1 HTML foundations & semantic markup

THEORY

Semantic HTML means choosing elements for what the content *means*, not how it looks.

`<header>` is not "a bar at the top", it is "introductory content".

Why it matters (three standard exam answers): 1. **Accessibility** — screen readers build a navigable outline from landmarks. 2. **SEO** — search engines weight `<h1>` and `<article>` content. 3. **Maintainability** — `<nav>` tells the next developer more than `<div class="nav">`.

Document structure

```
<!DOCTYPE html>           <!-- triggers standards mode, NOT quirks mode -->
<html lang="en">           <!-- lang aids screen readers and translation -->
<head>
  <meta charset="utf-8">      <!-- must be first 1024 bytes -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>...</title>         <!-- required -->
</head>
<body> ... </body>
</html>
```

Block vs inline

Block	Inline
Starts on a new line	Flows within a line
Takes full available width	Takes only content width
width/height apply	width/height ignored
div p h1-h6 ul ol li table section	span a strong em img code

`inline-block` = flows inline, but accepts width/height/vertical margins.

Key semantic elements

Element	Means
<code><header></code>	Introductory content for page or section
<code><nav></code>	Major navigation block
<code><main></code>	Dominant content — only one per page , not nested in header/nav/footer
<code><article></code>	Self-contained, independently distributable
<code><section></code>	Thematic grouping, normally with a heading
<code><aside></code>	Tangentially related (sidebar)
<code><footer></code>	Closing info for page or section
<code><figure></code> / <code><figcaption></code>	Image with caption
<code><time datetime="2026-10-07"></code>	Machine-readable date

`<article>` vs `<section>` : if the content would still make sense pulled out and published on its own (a blog post, a product card), use `<article>` . Otherwise `<section>` .

Tables — only for tabular data, never layout.

```
<table>
  <caption>Academic record</caption>
  <thead><tr><th scope="col">Exam</th></tr></thead>
  <tbody><tr><td>EMIT</td></tr></tbody>
</table>
```

`scope="col"` / `scope="row"` tells a screen reader which header governs which cell.

Lists: `<ul>` unordered, `<ol>` ordered, `<dl>` / `<dt>` / `<dd>` for term–definition pairs.

IN YOUR WORK

Assignment 01 (CV) used `article`, `header`, `section`, `main`, `footer`, `address`, `time`, `dl/dt/dd`, `table/thead/tbody/caption` . **Assignment 12** used `<caption>` and `scope="col"` in the student table.

PROBABLE QUESTIONS

- *Define semantic HTML and give three benefits.* Choosing elements for what content means, not how it looks. Benefits: accessibility, SEO, maintainability.
 - *Difference between `<section>` and `<div>` ?* `<section>` carries meaning and belongs in the outline; `<div>` is a styling hook with no meaning.
 - *Why `<!DOCTYPE html>` ?* Standards mode; without it browsers use quirks mode and the box model behaves differently.
 - *Block vs inline with examples.* Block starts on a new line and fills the width, and respects width/height: `div`, `p`, `h1`, `ul`, `section`. Inline flows within a line and ignores width/height: `span`, `a`, `strong`, `em`.
 - *Write the skeleton of a valid HTML5 page.* `<!DOCTYPE html>` , `<html lang>` , `<head>` with `charset` + `viewport` + `title`, then `<body>` .
 - *What does `alt` do?* Text alternative for screen readers and for when the image fails to load. Decorative images take `alt=""` .
-

2 Forms & input validation

THEORY

```
<form action="/submit" method="post">
```

GET vs POST — classic exam question:

	GET	POST
Data location	URL query string	Request body
Visible in address bar	Yes	No
Length limit	~2048 chars	Effectively none
Bookmarkable / cached	Yes	No
Use for	Searches, filters (idempotent)	Logins, uploads, anything that changes state

Labels — every control needs one:

```
<label for="email">Email</label>
<input type="email" id="email" name="email">
```

`for` must match the input's `id`. Clicking the label focuses the input, and screen readers announce it. **`name` is what gets submitted; `id` is for `for` /CSS/JS.** Without `name`, the field is not sent.

Input types: `text` `password` `email` `number` `date` `checkbox` `radio` `submit` `reset` `file` `tel` `url` `range` `color` `hidden`

Radio buttons sharing one `name` become mutually exclusive — that is *why* the shared name matters.

HTML5 validation attributes: `required`, `minlength`, `maxlength`, `min`, `max`, `step`, `pattern`, `placeholder`, `readonly`, `disabled`, `autocomplete`

`placeholder` is a **hint, not a label** — it disappears on typing.

Grouping: `<fieldset>` + `<legend>` (e.g. a group of radio buttons). `<select>` with `<option>`; `<datalist>` gives a free-text field with suggestions.

Client vs server validation: client-side is for user convenience and can be bypassed (DevTools, curl). Server-side is the real security boundary. **Always do both.**

IN YOUR WORK

Assignment 02 used text, password, email, number, date, checkbox, radio, submit, reset, plus `required/minlength/maxlength/min/max/pattern/title` . **A08/A09/A10/A11** added JS validation on top (`trim()` , empty checks, duplicates).

PROBABLE QUESTIONS

- *GET vs POST.* GET puts data in the URL, is limited in length, bookmarkable and cacheable, for reads. POST puts data in the request body, has no practical limit, is not cached, for anything that changes state.
 - *Purpose of `<label for>` ?* Accessibility + enlarged click target.
 - *Difference between `name` and `id` ?* `name` submits, `id` identifies.
 - *Why do radio buttons need the same `name` ?* It makes them one group so only one can be chosen.
 - *List five input types with purposes.* text (free text), email (format check), number (numeric with min/max), date (date picker), radio (one choice from a group), checkbox (independent on/off).
 - *Is client-side validation enough?* No — explain bypass + server-side.
-

3 CSS fundamentals & selectors

THEORY

Three ways to apply CSS

Method	Syntax	Notes
Inline	<code><p style="..."></code>	Highest specificity, unmaintainable, avoid
Internal	<code><style></code> in <code><head></code>	Single page only
External	<code><link rel="stylesheet" href="style.css"></code>	Preferred — caching, reuse, separation of concerns

Selectors

```
*          /* universal */
p          /* type */
.card      /* class – reusable */
#header    /* id – unique per page */
[type="text"] /* attribute */
```

Combinators (common exam question):

Combinator	Syntax	Meaning
Descendant	<code>A B</code>	B anywhere inside A
Child	<code>A > B</code>	B is a direct child of A
Adjacent sibling	<code>A + B</code>	B immediately follows A, same parent
General sibling	<code>A ~ B</code>	Any B after A, same parent

Pseudo-classes (a *state*): `:hover` `:focus` `:active` `:visited` `:first-child` `:last-child` `:nth-child(n)` `:not()` `:checked` `:disabled` `:focus-within`

Pseudo-elements (a *part*, written with `::`): `::before` `::after` `::first-line` `::first-letter`

`::before` / `::after` require a `content` property or they do not render.

Specificity — calculate as (inline, id, class, type):

Selector	Value
p	0,0,0,1
.card	0,0,1,0
#nav	0,1,0,0
#nav .item a	0,1,1,1
style="..."	1,0,0,0
!important	overrides everything (avoid)

Equal specificity → **last rule wins** (that's the "cascade").

Inheritance: text properties inherit (color , font-family , line-height); box properties do not (margin , padding , border , width).

Units

Unit	Relative to
px	Absolute
%	Parent
em	Parent's font size (compounds when nested)
rem	Root font size (no compounding — safer)
vw / vh	1% of viewport width/height
fr	Fraction of free space (Grid only)

CSS variables

```
:root { --navy: #14293f; }  
.header { color: var(--navy); }
```

Defined on `:root` so they are globally available; change once, applies everywhere.

IN YOUR WORK

Every assignment from 03 onwards uses an external stylesheet and `:root` variables. A05 demonstrates all four combinators explicitly.

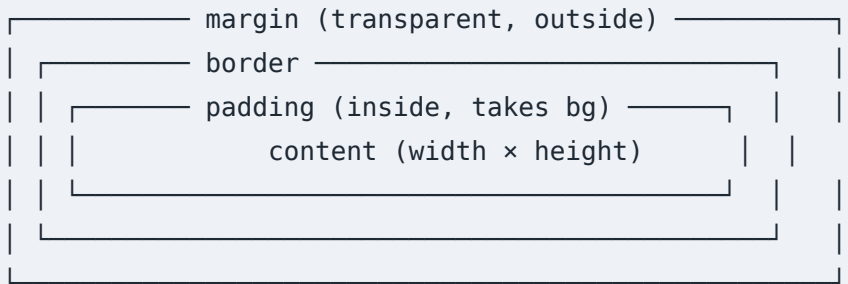
PROBABLE QUESTIONS

- *Three ways to include CSS; which is best and why?* Inline, internal `<style>`, external `<link>`. External is best: cached across pages, reusable, keeps structure separate from presentation.
 - *Explain the four combinators with examples.* `A B` descendant (any depth), `A > B` direct child, `A + B` immediately next sibling, `A ~ B` any later sibling.
 - *Calculate specificity of `#nav ul li a:hover`.* → 0,1,1,3 (`:hover` counts as a class)
 - *Pseudo-class vs pseudo-element.* A pseudo-class targets a state (`:hover`, `:first-child`), single colon. A pseudo-element targets a part of the element (`::before`, `::first-line`), double colon.
 - *em vs rem.* `em` is relative to the parent's font size and compounds when nested; `rem` is relative to the root font size and does not compound.
 - *What does "cascading" mean?* Rules combine by origin, specificity, then source order.
-

4 The box model

THEORY

Every element is a rectangle of four layers, inside out:



box-sizing — the single most examinable idea here

```
/* content-box (default) */
width: 300px; padding: 20px; border: 5px solid;
/* rendered width = 300 + 20+20 + 5+5 = 350px ← surprising */

/* border-box */
* { box-sizing: border-box; }
width: 300px; padding: 20px; border: 5px solid;
/* rendered width = 300px exactly; content shrinks to 250px */
```

Be ready to **compute a rendered width both ways**. Margin is *never* included in either calculation.

Shorthand order is clockwise from top:

```
margin: 10px; /* all four */
margin: 10px 20px; /* vertical | horizontal */
margin: 10px 20px 30px; /* top | horizontal | bottom */
margin: 10px 20px 30px 40px; /* top right bottom left (TRouBLE) */
```

Margin collapsing — adjacent *vertical* margins merge into the larger of the two; 30px below + 20px above = **30px gap, not 50px**. Horizontal margins never collapse. `margin: 0 auto` horizontally centres a block that has a set width.

overflow : `visible` (default) | `hidden` | `scroll` | `auto`

`display :` block | inline | inline-block | none | flex | grid `display: none` removes from layout entirely; `visibility: hidden` keeps the space.

IN YOUR WORK

Assignment 04 was built entirely on this: global `border-box`, a margin/padding experiment, and `overflow-x: hidden`.

PROBABLE QUESTIONS

- *Draw and label the box model.* Content in the middle, then padding, then border, then margin on the outside.
 - *An element has `width:200px; padding:10px; border:2px`. Total width under each `box-sizing` ?* → content-box: $200+20+4 = 224\text{px}$; border-box: **200px**
 - *Why set `box-sizing: border-box` globally?* Declared width = rendered width, so layout maths stays predictable.
 - *What is margin collapsing?* Adjacent vertical margins merge into the larger of the two rather than adding. 30px below plus 20px above gives a 30px gap. Horizontal margins never collapse.
 - *`display:none` vs `visibility:hidden`.* `display:none` removes the element from layout entirely; `visibility:hidden` hides it but keeps its space reserved.
 - *How do you centre a block horizontally?* `margin: 0 auto` with a width.
-

5 Flexbox

THEORY

One-dimensional layout — a row *or* a column. (Grid is two-dimensional.)

Axes: `flex-direction` sets the **main axis**; the **cross axis** is perpendicular. With `row`, main = horizontal. With `column`, main = vertical — so `justify-content` then controls *vertical* placement. Examiners love this.

Container properties

Property	Values
<code>display</code>	<code>flex</code> <code>inline-flex</code>
<code>flex-direction</code>	<code>row</code> <code>row-reverse</code> <code>column</code> <code>column-reverse</code>
<code>flex-wrap</code>	<code>nowrap</code> <code>wrap</code> <code>wrap-reverse</code>
<code>justify-content</code>	<code>flex-start</code> <code>flex-end</code> <code>center</code> <code>space-between</code> <code>space-around</code> <code>space-evenly</code>
<code>align-items</code>	<code>stretch</code> <code>flex-start</code> <code>flex-end</code> <code>center</code> <code>baseline</code>
<code>align-content</code>	aligns multiple lines (needs wrapping)
<code>gap</code>	spacing between items

`justify-content` = **main** axis. `align-items` = **cross** axis.

Item properties

Property	Meaning
<code>flex-grow</code>	Share of extra space (default 0)
<code>flex-shrink</code>	How it shrinks (default 1)
<code>flex-basis</code>	Starting size before grow/shrink
<code>flex: 1 1 200px</code>	shorthand grow/shrink/basis
<code>align-self</code>	Override <code>align-items</code> for one item
<code>order</code>	Visual reorder (does not change DOM/tab order)

Perfect centring:

```
.box { display: flex; justify-content: center; align-items: center; }
```

IN YOUR WORK

A05 nav bar (`display:flex` , `flex-wrap` , nested flex containers). **A08–A12** use flex for form rows, card metadata and button groups.

PROBABLE QUESTIONS

- *Flexbox vs Grid — when to use each?* Flex = 1D, content-driven, good for a nav bar or button row. Grid = 2D, layout-driven, good for a page skeleton.
 - `justify-content` vs `align-items`. `justify-content` positions items along the main axis; `align-items` positions them along the cross axis.
 - *What does `flex: 1` mean?* `flex-grow:1`; `flex-shrink:1`; `flex-basis:0` — equal share.
 - *Centre a div both ways using flexbox.* `display:flex; justify-content:center; align-items:center` on the parent.
 - *What changes if `flex-direction: column`?* The axes swap.
-

6 Positioning & dropdown menus

THEORY

position	Behaviour
static	Default. <code>top/left</code> ignored
relative	Offset from its normal spot ; original space preserved
absolute	Removed from flow; positioned against nearest positioned ancestor
fixed	Removed from flow; positioned against the viewport (stays on scroll)
sticky	Relative until a scroll threshold, then fixed

The dropdown pattern — the core of Assignment 06:

```
.dropdown      { position: relative; } /* ① containing block */
.dropdown-menu { position: absolute;  /* ② out of flow */
                top: 100%; left: 0;   /* ③ directly below parent */
                display: none;        /* ④ hidden */
                z-index: 1000; }      /* ⑤ above page content */
.dropdown:hover > .dropdown-menu { display: block; } /* ⑥ reveal */
```

Why each line matters: - `relative` on the parent makes it the reference for the absolute child. Without it, the menu positions against the whole page. - `absolute` takes the panel out of flow, so opening it **does not push the other nav items sideways**. - `top: 100%` = "100% of the parent's height down" = immediately below it. - `z-index` needs a *positioned* element to work at all.

Second level opens sideways with `left: 100%` instead of `top: 100%`.

Stacking context: `z-index` only applies to positioned elements. A new stacking context is created by `position` + `z-index`, and also by `opacity < 1`, `transform`, `filter`. A child can never escape its parent's stacking context — a common source of "my z-index: 9999 doesn't work".

IN YOUR WORK

Assignment 06 — two-level CSS-only dropdown, 15 pages, no JavaScript.

PROBABLE QUESTIONS

- Compare the five `position` values. static (default, offsets ignored), relative (offset from normal spot, space kept), absolute (out of flow, against nearest positioned ancestor), fixed (out of flow, against the viewport), sticky (relative until a scroll threshold, then fixed).
 - Why does the dropdown parent need `position: relative`? It makes the parent the containing block, so the absolutely positioned menu is placed against it instead of against the page.
 - Write CSS for a hover dropdown. Parent `position: relative`; menu `position: absolute; top: 100%; display: none; z-index: 1000`; then `.dropdown:hover > .dropdown-menu { display: block; }`.
 - What is `z-index` and when does it not work? Only on positioned elements; also constrained by stacking contexts.
 - How do you make a submenu open to the right? `left: 100%; top: 0`.
 - `absolute` vs `fixed`. Both leave the normal flow. Absolute positions against the nearest positioned ancestor and scrolls with the page; fixed positions against the viewport and stays put when scrolling.
-

7 CSS Grid

THEORY

Two-dimensional — rows and columns together.

```
.layout {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr) minmax(240px, 300px);  
  gap: 22px;  
  grid-template-areas:  
    "nav      nav      nav      nav"  
    "feature feature feature sidebar"  
    "col-one col-two col-three sidebar"  
    "footer  footer  footer  footer";  
}  
.site-nav { grid-area: nav; }  
.sidebar  { grid-area: sidebar; }
```

Sizing functions

Function	Meaning
1fr	One share of leftover space
repeat(3, 1fr)	Three equal columns
minmax(240px, 300px)	Never below 240, never above 300
auto-fit / auto-fill	Fit as many tracks as will fit
repeat(auto-fit, minmax(200px, 1fr))	The classic responsive grid, no media query

Placement: by **named area** (readable, self-documenting) or by **line number** (`grid-column: 1 / 3`, meaning from line 1 to line 3 = spanning 2 columns). Note lines are 1-indexed and you count *lines*, not columns.

Advantage of named areas: a media query only has to redraw the area map — no element needs new positioning rules.

```
@media (max-width: 900px) {  
  .layout {  
    grid-template-columns: repeat(2, 1fr);  
    grid-template-areas: "nav nav" "feature feature" "col-one col-two" "footer footer";  
  }  
}
```

Alignment: `justify-items` / `align-items` (items in their cell), `justify-content` / `align-content` (the whole grid in the container). `place-items: center` is shorthand for both.

Liquid vs fixed layout: liquid uses `fr` / `%` / `minmax` so it fills any viewport; fixed uses `px` and leaves gaps or overflows.

IN YOUR WORK

Assignment 07 — liquid layout with named areas, redrawn at 900px and 600px. **Assignment 11** — three kanban lanes with `repeat(3, 1fr)`.

PROBABLE QUESTIONS

- *Grid vs Flexbox.* Grid is two-dimensional, for rows and columns together, layout-driven. Flexbox is one-dimensional, a single row or column, content-driven.
 - *What is `fr`?* A fraction of the **remaining** free space after fixed tracks.
 - *Explain `grid-template-areas` and its benefit.* It names regions in an ASCII-style map and places elements with `grid-area`. The benefit is that a media query only redraws the map, so no element needs new positioning rules.
 - *What does `repeat(auto-fit, minmax(200px, 1fr))` do?* It creates as many columns as will fit, each at least 200px and sharing leftover space equally. A responsive grid with no media query.
 - *Make an element span two columns.* `grid-column: span 2`.
 - *`gap` vs margins?* `gap` applies only *between* tracks, with no edge margins and no collapsing.
-

8 JavaScript basics & functions

THEORY

`var` / `let` / `const`

	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function	Block	Block
Redeclare	Yes	No	No
Reassign	Yes	Yes	No
Hoisted	Yes, as <code>undefined</code>	Yes, but in the TDZ	Same

`const` prevents *reassignment*, not mutation — `const a = [1]; a.push(2)` is legal.

Three ways to write a function

```
// 1. Declaration – fully hoisted, callable before its definition
function add(a, b) { return a + b; }
```

```
// 2. Expression – only the variable is hoisted, not the function
var isNumeric = function (text) { ... };
```

```
// 3. Arrow – concise; no own `this`, `arguments`, or `prototype`
const formatSum = (a, symbol, b) => a + ' ' + symbol + ' ' + b;
```

Hoisting — the classic trick question:

```
sayHi(); // works: declarations are hoisted whole
function sayHi() {}
```

```
sayLater(); // TypeError: sayLater is not a function
var sayLater = function () {};
```

Arrow functions and `this` — the other classic. An arrow has no `this` of its own, it inherits from the enclosing scope. So an arrow is **wrong** when you need `this` to mean the element:

```
btn.addEventListener('click', function () { this; /* the button */ });
btn.addEventListener('click', () => { this; /* outer scope – not the button */ });
```

Implicit return: `x => x * 2` returns automatically; with braces you must write `return`.

Type coercion

	<code>==</code>	<code>===</code>
Compares	Value after coercion	Value and type
<code>5 == "5"</code>	true	false
<code>0 == false</code>	true	false
<code>null == undefined</code>	true	false

Always prefer `===`.

Falsy values (memorise all 7): `false`, `0`, `""`, `null`, `undefined`, `NaN`, `0n`. Everything else is truthy — including `[]` and `{}`.

Useful methods

- Strings: `trim()` `toLowerCase()` `split()` `slice()` `includes()` `padStart()`
- Arrays: `push()` `pop()` `shift()` `unshift()` `splice()` `slice()` `forEach()` `map()` `filter()` `find()` `some()` `every()` `indexOf()` `includes()`
- `map` returns a new array; `forEach` returns `undefined`.
- Conversion: `Number(x)`, `parseInt(x, 10)`, `String(x)`, `Number.isFinite(x)`

IN YOUR WORK

A08 validation helpers. **A09** deliberately used all three function forms: declarations (`add`, `subtract`, `multiply`, `divide`), expressions (`showError`, `validate`), arrows (`isNumeric`, `formatNumber`, `handlers`).

PROBABLE QUESTIONS

- *Difference between* `var`, `let` and `const`. `var` is function-scoped and hoisted as `undefined`; `let` is block-scoped and reassignable; `const` is block-scoped and cannot be reassigned,

though objects it holds can still be mutated.

- *What is hoisting? Give an example where a declaration works and an expression fails.*
Declarations are moved to the top of their scope. Calling `sayHi()` before `function sayHi(){}` works; calling `fn()` before `var fn = function(){}` throws a `TypeError` because only the variable is hoisted.
 - *Declaration vs expression vs arrow.* A declaration is fully hoisted and callable early. An expression assigns a function to a variable and must be defined first. An arrow is concise, has no own `this`, `arguments` or `prototype`.
 - *Why can't you always use an arrow function as an event handler?* No own `this`.
 - `==` vs `===` with examples. `==` coerces types, so `5 == "5"` and `0 == false` are true. `===` compares value and type, so both are false. Prefer `===`.
 - *List the falsy values.* `false`, `0`, empty string, `null`, `undefined`, `NaN` and `0n`. Everything else is truthy, including empty arrays and objects.
 - *What does `trim()` do and why use it before validating?* It removes leading and trailing whitespace, so an entry of only spaces is correctly treated as empty.
-

9 DOM manipulation

THEORY

The **DOM** is the browser's tree-shaped object model of the page. It is an API over the document, not the HTML text itself.

Selecting

Method	Returns
<code>getElementById(id)</code>	One element or <code>null</code>
<code>getElementsByClassName(c)</code>	Live HTMLCollection
<code>getElementsByTagName(t)</code>	Live HTMLCollection
<code>querySelector(sel)</code>	First match (any CSS selector) or <code>null</code>
<code>querySelectorAll(sel)</code>	Static NodeList

Live means it updates automatically as the DOM changes; *static* is a snapshot. `querySelectorAll` returns static — a frequent exam point.

Creating and inserting

```
const li = document.createElement('li');
li.textContent = 'Learn JavaScript';    // safe
li.classList.add('task-item');
parent.appendChild(li);                 // or prepend, insertBefore, append
```

Content properties

Property	Behaviour
<code>textContent</code>	Plain text; escapes HTML — safe
<code>innerHTML</code>	Parses as markup — XSS risk with user input
<code>innerText</code>	Like <code>textContent</code> but respects CSS visibility, slower

Why this matters: if a user types `<img src=x onerror=alert(1)>`, `innerHTML` executes it, `textContent` displays it as text. Your to-do and kanban apps use `textContent` for exactly this reason, and I verified the escaping.

Attributes vs properties: `getAttribute('href')` reads the HTML source value; `el.href` gives the resolved absolute URL. `dataset.id` maps to `data-id`.

Classes: `classList.add()` `.remove()` `.toggle()` `.contains()` `.replace()`

`classList.toggle('done')` adds the class if absent, removes if present, and **returns a boolean** of the new state:

```
const nowDone = item.classList.toggle('completed');
```

Removing: `element.remove()` (modern) or `parent.removeChild(child)` (old).

Traversal: `parentNode`, `children`, `firstElementChild`, `nextElementSibling`, `closest(selector)` (walks *up* to the nearest matching ancestor).

IN YOUR WORK

A10 used all of these: `createElement`, `textContent`, `classList.add/toggle`, `appendChild`, `.remove()`, `dataset.index`. **A11** added `closest('.card')` and `replaceChild` for inline editing.

PROBABLE QUESTIONS

- *What is the DOM?* The Document Object Model: the browser's tree-shaped object representation of the page, which JavaScript can read and change.
- `getElementById` vs `querySelector`. `getElementById` takes only an id and is slightly faster; `querySelector` accepts any CSS selector and returns the first match.
- `innerHTML` vs `textContent` — *which is safer and why?* `textContent` is safer because it escapes markup. `innerHTML` parses its input, so user-typed HTML such as an `onerror` image would execute, which is an XSS risk.
- *Write JS to create an `<li>` with text and a class, then append it to a `<ul>`.*

```
const li = document.createElement('li'); li.textContent = 'Task'; li.classList.add('item'); list.appendChild(li);
```
- *What does `classList.toggle()` return?* A boolean giving the new state: true if the class is now present, false if it was removed.

- *How do you remove an element?* `element.remove()` , or the older `parent.removeChild(child)` .
 - *Live vs static collection.* `getElementsByClassName` returns a live `HTMLCollection` that updates as the DOM changes; `querySelectorAll` returns a static `NodeList` snapshot.
-

10 Events & event delegation

THEORY

Registering:

```
element.addEventListener('click', handler); // preferred – many handlers
element.onclick = handler; // only one, overwrites
<button onclick="doIt()"> // inline – avoid
```

Event object: `event.target` (what was actually clicked), `event.currentTarget` (what the listener is attached to), `event.preventDefault()`, `event.stopPropagation()`.

target vs **currentTarget** is a favourite exam question. Click a `<span>` inside a `<button>` with the listener on the button: `target` is the span, `currentTarget` is the button.

Propagation — three phases: 1. **Capturing** — window → target (listener needs `{capture: true}`) 2. **Target** — at the element 3. **Bubbling** — target → window (**the default**)

Event delegation — the big idea:

```
// One listener on the parent, instead of one per child
list.addEventListener('click', function (event) {
  const item = event.target.closest('.task-item');
  if (!item) return;
  if (event.target.classList.contains('btn-delete')) { ... }
});
```

Three advantages: 1. **Works for elements added later** — no rebinding after a re-render. 2. **Less memory** — one listener instead of hundreds. 3. **Simpler code** — no cleanup when elements are destroyed.

This is *why* your to-do and kanban apps attach one listener to the `<ul>` /board rather than to each card: `render()` destroys and rebuilds every card, so per-card listeners would be lost each time.

preventDefault() stops the browser's default action (form submit reloading, link navigating).

stopPropagation() stops the bubble. They are different.

Common events: `click` `dblclick` `submit` `input` `change` `focus` `blur` `keydown` `keyup` `mouseenter` `mouseleave` `DOMContentLoaded` `load` `dragstart` `dragover` `drop`

`input` fires on every keystroke; `change` fires when the value is committed (blur, or selection made).

Form submit:

```
form.addEventListener('submit', function (e) {  
  e.preventDefault();      // without this the page reloads and state is lost  
  addTask();  
});
```

Using a real `<form>` gets **Enter-key submission for free** — that is how Assignment 10's "Extension 1" worked without any keydown handler.

IN YOUR WORK

A10/A11 — delegated `click` on the list/board, `event.target` dispatch, `preventDefault()` on submit, `keydown` for Enter/Escape in edit mode.

PROBABLE QUESTIONS

- *What is event bubbling? Name the three phases.* Bubbling is the event travelling from the target up to the root. The phases are capturing (root to target), target, then bubbling (target to root).
 - *What is event delegation and why use it?* Attaching one listener to a parent and using `event.target` to identify the child. It keeps working for elements added later, uses less memory, and needs no cleanup when children are destroyed.
 - `event.target` VS `event.currentTarget`. `target` is the element actually clicked; `currentTarget` is the element the listener is attached to.
 - `preventDefault()` VS `stopPropagation()`. `preventDefault` cancels the browser's default action, such as a form reload. `stopPropagation` stops the event travelling further up the tree.
 - *Why call `preventDefault()` on form submit?* Without it the page reloads, which discards the JavaScript state you just built.
 - *Three ways to attach a handler; which is best?* Inline `onclick` attribute, the `element.onclick` property, and `addEventListener`. The last is best: it allows many handlers and finer control.
-

11 Drag and drop

THEORY

HTML5 drag-and-drop needs **three things**: a draggable source, a drop target that *allows* dropping, and data passed between them.

```
element.draggable = true; // ① opt in, or draggable="true"
```

The event sequence

Event	Fires on	Job
dragstart	Source	Store data, add dragging style
dragover	Target	preventDefault() to allow the drop
dragenter / dragleave	Target	Highlight / un-highlight
drop	Target	Read data, perform the move
dragend	Source	Cleanup, fires even if cancelled

The counter-intuitive bit:

```
board.addEventListener('dragover', function (event) {  
  event.preventDefault(); // elements REJECT drops by default;  
                          // this cancels the rejection  
});
```

Without `preventDefault()` in `dragover`, **drop never fires**. You are preventing the *rejection*, not the drop — guaranteed exam question.

dataTransfer carries data for the drag's lifetime, **strings only** — which is why you store an ID and look the object up again:

```
event.dataTransfer.setData('text/plain', card.dataset.id); // dragstart  
const id = Number(event.dataTransfer.getData('text/plain')); // drop
```

Key principle: the drop handler does **not** move the DOM node. It updates the data (`task.lane = 'done'`) and calls `render()` . The array is the source of truth; the DOM is a picture of it.

IN YOUR WORK

Assignment 11 — kanban with all five events, delegated on the board.

PROBABLE QUESTIONS

- *List the drag-and-drop events in order.* `dragstart` on the source, then `dragover` and `dragenter/dragleave` on the target, then `drop`, and finally `dragend` on the source.
 - *Why is `preventDefault()` needed in `dragover`?* Elements reject drops by default. Calling `preventDefault` cancels that rejection and marks the element a valid drop target; without it the drop event never fires.
 - *How is data passed from source to target?* `dataTransfer` , strings only.
 - *Why store an ID rather than the element?* `dataTransfer` can only hold strings, so you store the id and look the object up again in the drop handler.
 - *Which event fires even when a drag is cancelled?* `dragend` .
-

12 Objects & classes

THEORY

Object literal

```
const student = { id: 1, name: 'Leanne' };
student.name;           // dot notation
student['name'];         // bracket – needed for dynamic keys
```

Constructor function (ES5)

```
function Student(id, name) { this.id = id; this.name = name; }
Student.prototype.greet = function () { return 'Hi ' + this.name; };
```

Class (ES6) — syntactic sugar over the same prototype mechanism:

```
class Student {
  constructor(id, name, username, email) { this.id = id; /* ... */ }

  static fromApi(record) { return new Student(record.id, ...); } // on the CLASS
  static isValid(record) { ... }

  mailto() { return 'mailto:' + this.email; } // on INSTANCES
  toString() { return this.name + ' (' + this.username + ')'; }
}
```

Class vs constructor function — three real differences: 1. A class **cannot be called without new** (throws `TypeError`); a constructor function silently pollutes the global object. 2. Class methods are **non-enumerable** (hidden from `for...in`). 3. Classes are **not hoisted** in a usable way (TDZ).

static vs instance: `static` belongs to the class (`Student.fromApi(...)`), instance methods belong to objects (`student.mailto()`). `student.fromApi` is `undefined`.

Why use a class for API data? (Assignment 12's rationale) - Drops fields you do not need (the API sends `address`, `phone`, `company`). - **One place to change** if the API renames a field — only `fromApi` edits. - Methods keep display logic on the object (`student.mailto()`).

this depends on call site: method call → the object; plain function → `undefined` in strict mode; arrow → enclosing scope.

JSON: `JSON.stringify(obj)` object → string, `JSON.parse(str)` string → object. **JSON is a string format, not an object.** `JSON.parse` throws on malformed input, so wrap it in `try/catch`.

IN YOUR WORK

A12 — `class Student` with `constructor`, two statics, two instance methods. **A10/A11** — `localStorage` with `JSON.stringify / parse` in `try/catch`.

PROBABLE QUESTIONS

- *Define a class with a constructor and a method.* `class Student { constructor(id, name) { this.id = id; this.name = name; } greet() { return 'Hi ' + this.name; } }`
 - *static vs instance method.* A static method is called on the class itself, such as `Student.fromApi(...)`. An instance method is called on an object, such as `student.mailto()`.
 - *Class vs constructor function — give differences.* A class cannot be called without `new`, its methods are non-enumerable, and it is not useably hoisted. A constructor function allows all three and can silently pollute the global object.
 - *What does new do?* Creates an empty object, sets its prototype, binds `this`, runs the constructor, returns the object.
 - *JSON.parse vs JSON.stringify.* `stringify` turns an object into a JSON string for storage or sending; `parse` turns a JSON string back into an object.
 - *Why wrap JSON.parse in try/catch?* It throws on malformed input, so corrupt stored data would break the whole script without a catch.
-

13 Asynchronous JS: fetch & Promises

THEORY

Why async? JavaScript is **single-threaded**. A synchronous network call would freeze the entire page. Async work is handed to the browser, and the callback is queued for when it finishes.

Event loop (one-line version): call stack runs code; completed async work waits in the task queue; the loop moves queued callbacks onto the stack only when it is empty. Microtasks (promises) jump ahead of macrotasks (`setTimeout`).

Promise states: `pending` → `fulfilled` or `rejected` . Once settled it never changes.

The chain — Assignment 12's core:

```
fetch(API_URL)
  .then(function (response) {
    if (!response.ok) { // ← ESSENTIAL
      throw new Error('Request failed with status ' + response.status);
    }
    return response.json(); // returns another promise
  })
  .then(function (data) {
    const students = data.filter(Student.isValid).map(Student.fromApi);
    showStudents(students);
  })
  .catch(function (error) { // any failure above
    setStatus('Could not load: ' + error.message, 'error');
  })
  .finally(function () { // always, either way
    loadBtn.disabled = false;
  });
```

The single most important exam point:

fetch() does not reject on HTTP errors. A 404 or 500 still *resolves*. `fetch` only rejects on a **network failure** (no connection, DNS failure, CORS block). You must check `response.ok` and throw manually.

Without that check, a 500 falls through and `.json()` fails with a confusing parse error instead of a clear status message.

What each link does

Method	Runs when	Returns
<code>.then(fn)</code>	Previous step fulfilled	A new promise
<code>.catch(fn)</code>	Any earlier step rejected or threw	A new promise
<code>.finally(fn)</code>	Always — success or failure	Passes the result through

`.finally()` is for cleanup that must happen regardless: re-enabling a button, hiding a spinner, saying "request completed". It receives **no arguments** — it does not know whether things succeeded.

Response methods: `.json()`, `.text()`, `.blob()` — each returns a *promise*, which is why there are two `.then()` s.

Returning inside `.then()` matters:

```
return showStudents(students).then(...) // chain waits for the rows to render
showStudents(students);                 // chain does NOT wait – bug
```

async / await — sugar over the same promises:

```
async function load() {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error('...');
    const data = await res.json();
  } catch (e) { ... } finally { ... }
}
```

An `async` function **always returns a promise**. `await` only works inside `async`.

The `forEach` trap (this bit you hit for real):

```
await items.forEach(async (item) => {      // x does NOT wait
  await wait(100);
});
```

`forEach` **ignores whatever its callback returns**, so awaiting inside it does not pause the loop — all callbacks start at once. Use a `for...of` loop or chain promises instead. I measured this in your Assignment 12: all 10 rows appeared at 25ms instead of staggering.

Callback hell → **promises** → **async/await** is the historical progression; promises solved deep nesting via chaining.

AJAX = updating part of a page without a full reload. `fetch` is the modern API; `XMLHttpRequest` is the older one.

CORS: a browser security rule blocking cross-origin requests unless the server sends `Access-Control-Allow-Origin`. JSONPlaceholder sends a permissive header, which is why your page works from `file://`.

IN YOUR WORK

Assignment 12 — full chain, `response.ok` check, four distinct error paths, `.finally()` with a visible completion line, and staggered row rendering via chained promises.

PROBABLE QUESTIONS

- *Why is async programming needed in JS?* Single thread; avoid blocking.
- *What are the three promise states?* Pending, then either fulfilled or rejected. Once settled the state never changes again.
- *Write a fetch with `.then()`, `.catch()` and `.finally()`.* `fetch(url).then(r => { if (!r.ok) throw new Error(r.status); return r.json(); }).then(showData).catch(showError).finally(hideSpinner);`
- *Does `fetch` reject on a 404?* **No** — explain `response.ok`.
- *Difference between `.then()` and `.finally()`.* `.then` runs only on success and receives the value; `.finally` runs on both success and failure, receives no arguments, and is for cleanup.
- *Why does `response.json()` need its own `.then()`?* It returns a promise.
- *Convert a promise chain to `async / await`.* Mark the function `async`, `await` each promise, and replace `.catch / .finally` with `try / catch / finally`.

- *What is AJAX? What is CORS?* AJAX is updating part of a page without a full reload. CORS is the browser rule blocking cross-origin requests unless the server sends an Access-Control-Allow-Origin header.
-

14 Quick-reference tables

MUST-MEMORISE NUMBERS & FACTS

Fact	Value
Falsy values	<code>false, 0, "", null, undefined, NaN, 0n</code>
Promise states	pending, fulfilled, rejected
Event phases	capturing, target, bubbling
Box model layers	content, padding, border, margin
Margin shorthand	top, right, bottom, left (clockwise)
<code><main></code> per page	Exactly one
Specificity order	inline > id > class > type
<code>fetch</code> rejects on	Network failure only — not HTTP errors
<code>dragover</code> needs	<code>preventDefault()</code>
<code>forEach</code> returns	<code>undefined</code> (ignores callback returns)

CSS PROPERTY → PURPOSE

Need	Property
Predictable widths	<code>box-sizing: border-box</code>
Centre a block	<code>margin: 0 auto</code>
Centre anything	<code>display:flex; justify-content:center; align-items:center</code>
Element above others	<code>position + z-index</code>
Hide but keep space	<code>visibility: hidden</code>
Hide completely	<code>display: none</code>
Responsive columns	<code>repeat(auto-fit, minmax(200px, 1fr))</code>
Screen-reader-only	<code>clip-path: inset(50%) + 1px size</code>

JS "WHICH ONE DO I USE"

Need	Use
Safe text insert	<code>textContent</code>
Element by CSS selector	<code>querySelector</code>
Handle clicks on many items	Delegation + <code>event.target</code>
Nearest matching ancestor	<code>closest()</code>
Toggle a state class	<code>classList.toggle()</code>
Persist across refresh	<code>localStorage + JSON.stringify</code>
Remove an element	<code>.remove()</code>
Always-run cleanup	<code>.finally()</code>

15 Exam strategy

Likely paper structure: short answers (definitions, comparisons) → code snippets (write CSS for X, write JS to do Y) → output prediction → one longer "explain and justify" question.

The comparisons most likely to appear — have a one-line answer ready for each: GET/POST · id/class · block/inline · margin/padding · content-box/border-box · relative/absolute · flex/grid · var/let/const · ==/=== · innerHTML/textContent · target/currentTarget · .then/.catch/.finally · declaration/expression/arrow

Code questions — write the comment too. Markers award method marks for showing you know *why* (`/* preventDefault stops the page reloading */`).

Three "gotcha" answers that score well: 1. `fetch` does not reject on 404/500 — check `response.ok` . 2. `dragover` needs `preventDefault()` or `drop` never fires. 3. `await` inside `forEach` does not wait.

If you forget a property name, describe the behaviour and name the concept. Partial credit is real.

Diagrams: the box model and the three event phases are quick to sketch and often worth easy marks.

From your own assignments, be ready to explain: - A04 — why global `border-box` - A06 — why the dropdown parent needs `position: relative` - A07 — why named grid areas simplify media queries - A09 — the three function forms and hoisting - A10 — why one delegated listener instead of per-item listeners - A11 — why the drop handler changes data rather than moving DOM nodes - A12 — why `response.ok` must be checked manually

Good luck. The strongest preparation is being able to open any one of your twelve assignments and explain every line in it.